

Tema 6: Model Context Protocol (MCP)

EBIS Business Techschool

Contenidos

- Limitaciones de las Tools
- ¿Qué es una herramienta?
- ¿Qué es MCP?
- Servidor MCP
- Cliente MCP
- MCPs y otras estrategias
- Q&A y cierre



Limitaciones de las Tools

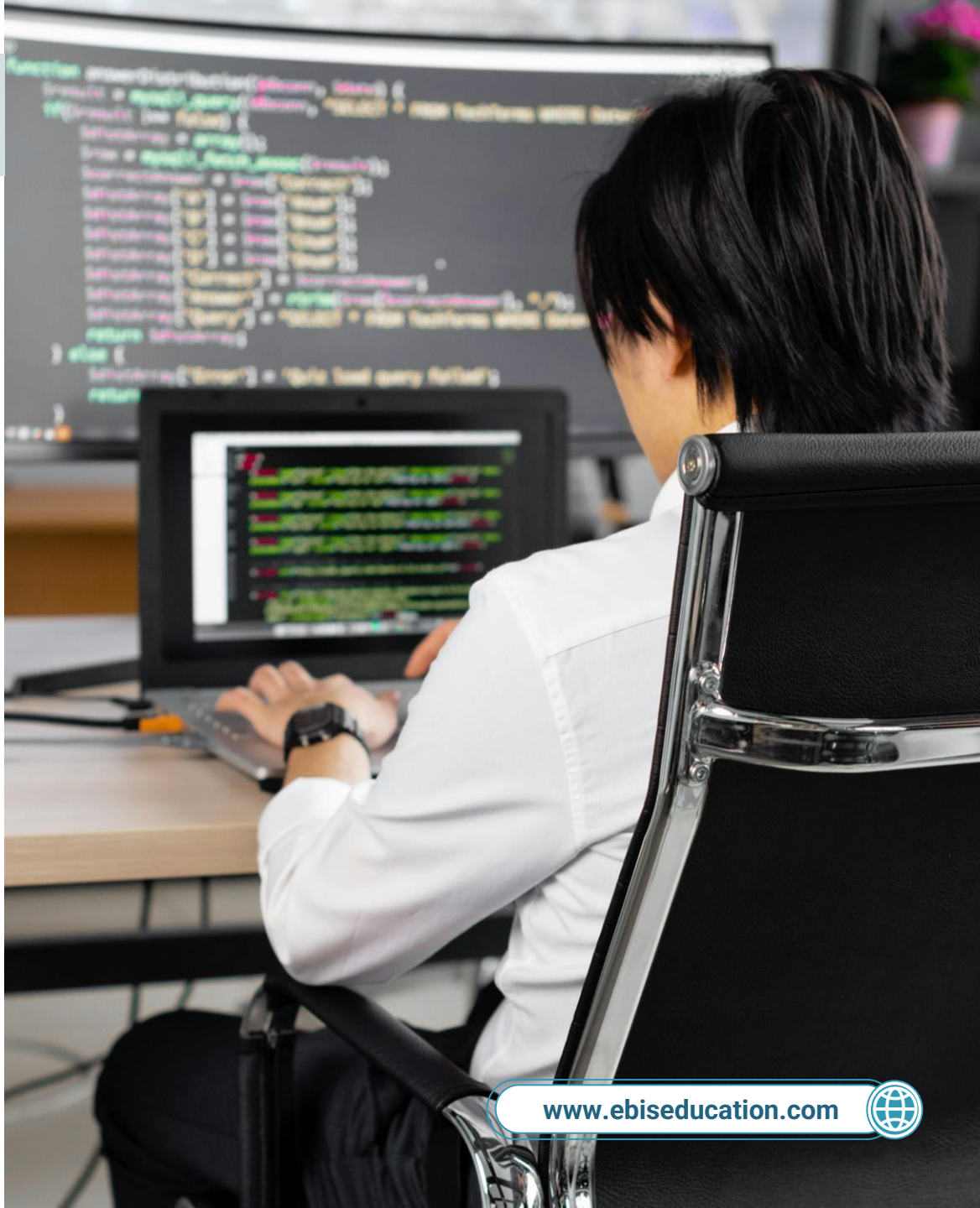


EBIS Business Techschool

La pregunta de partida

¿Cuál es el límite? ¿Cuántas conexiones podemos hacer para potencial nuestros agentes?

- Gestión de ficheros ✅
- Ejecución de commandos por terminal
- Conexión a una base de datos.
- Conectar un calendario (Google, Outlook)
- Envío de correos automáticos
- Búsqueda en una API externa
- ...



¿Qué es MCP y por qué existe?



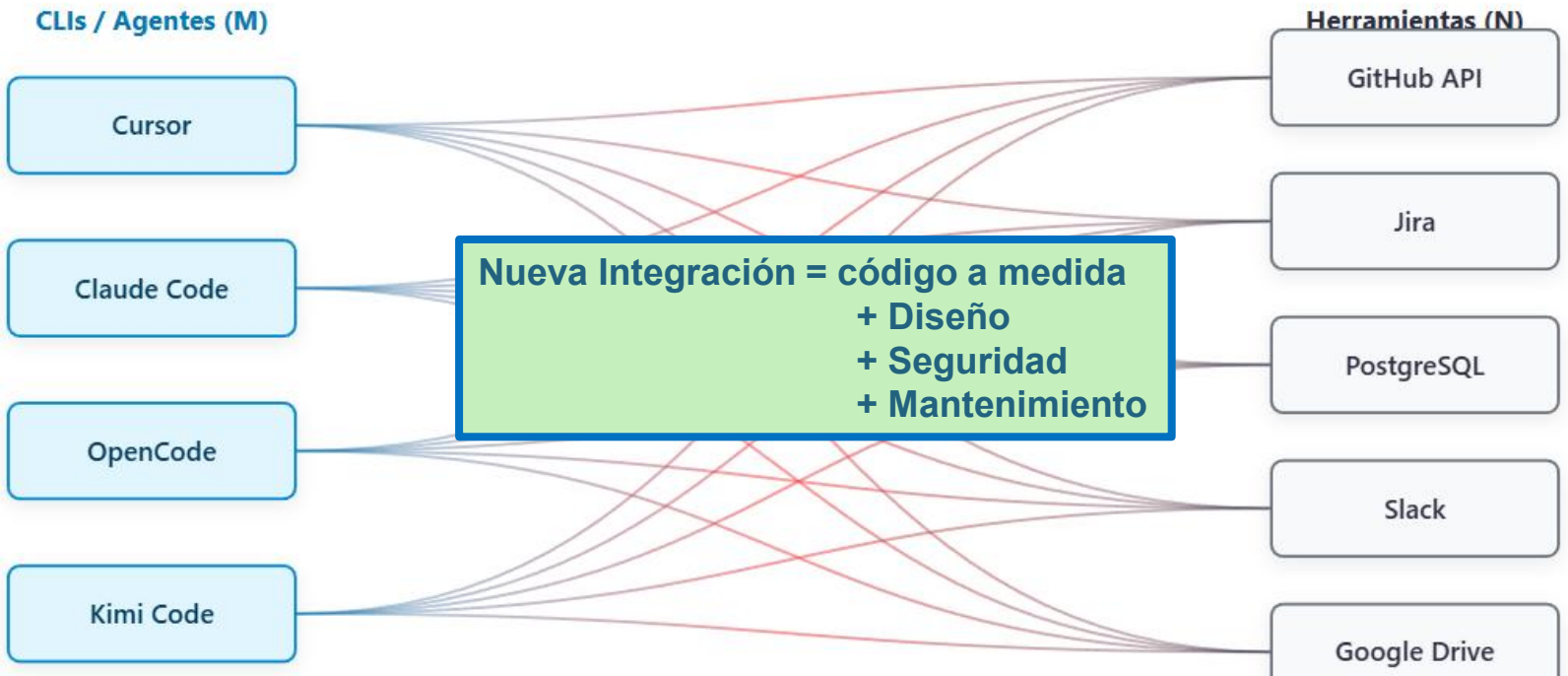
EBIS Business Techschool

El problema M×N

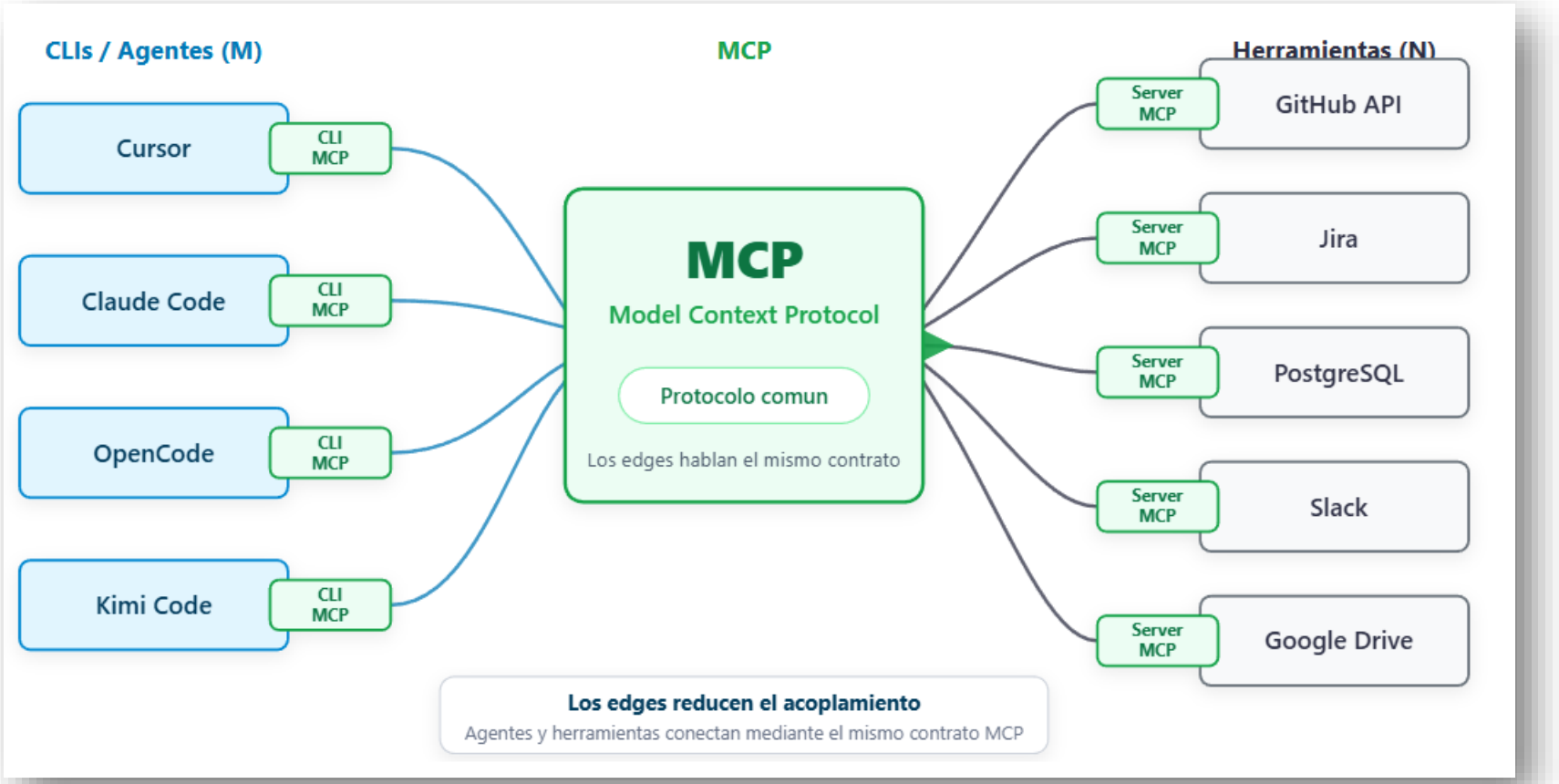
Sin estándar → MxN

Para cada **CLI Agentico:**

Agente (M) × Cada Herramienta (N) = Nueva Integración



¿Qué es MCP?



MCP convierte M×N en M+N

"El USB-C de las aplicaciones de IA"

¿Qué es MCP?

Protocolo abierto que estandariza cómo las aplicaciones de IA se conectan con fuentes de datos y herramientas externas.

- **Origen**: propuesto por Anthropic, noviembre 2024
- **Naturaleza**: especificación pública, SDKs en múltiples lenguajes
- **Adopción**: Actualmente es un estándar → Claude Code, Codex, Cursor, VSCode…

"Cualquiera puede implementar un servidor o un cliente MCP"

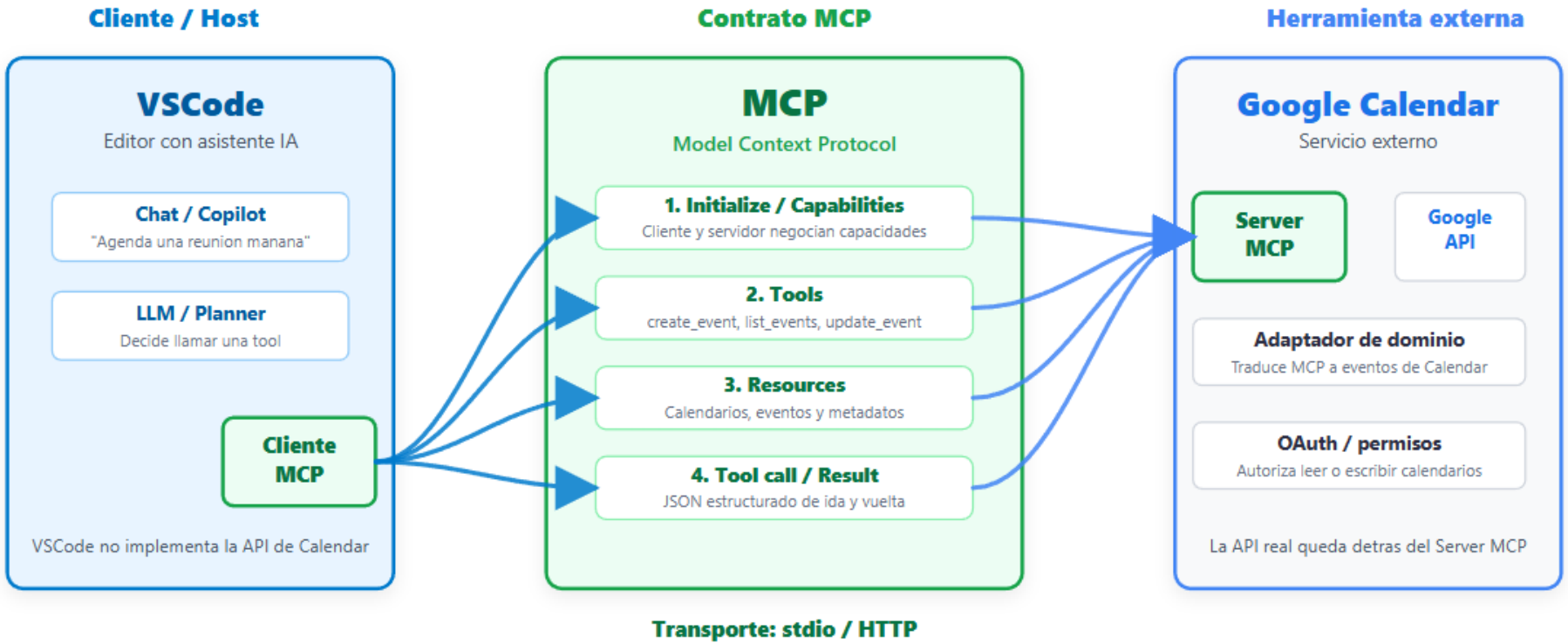
¿Qué es MCP?

EBIS Business Techschool

¿Cuándo se necesita MCP? ¿Cuándo tool nativa?

Pregunta	Si la respuesta es “sí”	Tendencia
¿Representa un **sistema externo con identidad propia**?	GitHub, Slack, Stripe	**MCP**
¿Debe **reutilizarse desde varios agentes o aplicaciones**?	Claude, Codex, Cursor, agente propio	**MCP**
¿Está **íntimamente ligada al runtime, sandbox o UI del agente**?	Shell, archivos locales, `ask_user`	**Tool nativa**
¿Es una **operación simple, pura o trivial**?	Calcular, parsear JSON, hacer diff de ficheros	**Tool nativa**

Arquitectura MCP



- **Host**: aplicación con la que interactúa el usuario (Claude Code, Codex, Cursor, tu app…)
- **Cliente MCP**: Vive dentro del host, mantiene conexión 1:1 con un servidor
- **Servidor MCP**: Expone `tools/function calls`, resources y prompts; usando un *idioma* estándar que entiende cualquier cliente MCP

Comunicación MCP

EBIS Business Techschool

Cliente y Servidor MCP se comunicación siguiendo un lenguaje universal común utilizando el protocolo **JSON-RPC 2.0**.

¿Qué es JSON-RPC?

Es un protocolo para comunicar sistemas donde el **cliente le pide al servidor ejecutar una función específica por su nombre**, enviándole los datos en formato JSON a través de una única ruta HTTP (usualmente un POST a /api).

A diferencia de REST, que se organiza en torno a "recursos", JSON-RPC **se enfoca en "acciones".**



Comunicación MCP

JSON-RPC está diseñado para funcionar de manera **asíncrona, desordenada o masiva**, el "id" es la etiqueta que le permite al cliente saber a qué pregunta corresponde cada respuesta del servidor.



Comunicación MCP

EBIS Business Techschool

Flujo de Comunicación **Versión: 2025-11-25**

1. HANDSHAKE INICIAL

1. **Petición**: El cliente inicia sesión enviando **`initialize`**, detallando sus capacidades y versión del protocolo.
2. **Respuesta**: El servidor confirma la versión y expone sus propias **capacidades**.
3. **Confirmación**: El Cliente envía **`notifications/initialized`** para confirmar el acuerdo.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2025-11-25",
    "capabilities": {
      "tools": {},
      "resources": {},
      "prompts": {}
    },
    "serverInfo": { "name": "mi-servidor-completo", "version": "1.0.0" }
  }
}
```

```
"capabilities": {
  "tools": {
    "listChanged": true
  },
  "resources": {
    "subscribe": true
  },
  "prompts": {}
},
```

*Dentro del objeto **`capabilities`** **no se envían las herramientas ni los datos**, sino únicamente indicadores sobre las `tools/resources/prompts`.*

Comunicación MCP

EBIS Business Techschool

Handshake Inicial: ¿Por qué existe y qué resuelve? Versión: 2025-11-25

1. Negociación de Versiones

Acuerdo de versiones: Cliente (Codex) y Servidor (tool MCP) pueden evolucionar en diferentes velocidades.

Antes de hablar **negocian en que versión común** del protocolo MCP que una a usar.

2. Capabilities

Tanto cliente cómo servidor comparten las **tools** y **resources** que se ofrecen el uno al otro a modo de “Discovery”.

3. Sincronización en local

En MCPs locales, nuestro agente arranca el servidor como un subproceso.

El handshake se asegura que todo está listo antes de recibir órdenes. Al ser una conexión privada entre procesos tener la sesión en memoria es la arquitectura más **simple**, **limpia** y **eficiente**.

Comunicación MCP

EBIS Business Techschool

Flujo de Comunicación

Versión: 2026-07-28 → stateless

1. Comunicación stateless
(Sin handshake)

1. **Sin sesión previa:** Se eliminan **`initialize`** y **`notifications/initialized`**. Ya no se abre una sesión en memoria; cualquier petición puede ser la primera.

2. **Metadatos por petición:** El cliente adjunta en cada llamada su carta de presentación: versión del protocolo y capacidades dentro del campo **`_meta`**.

3. **Descubrimiento bajo demanda**: Si el cliente necesita conocer previamente qué ofrece el servidor, invoca el método directo **`server/discover`**.

`server/discover`

```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2026-07-28",
    "capabilities": {
      "tools": {},
      "resources": {},
      "prompts": {}
    },
    "serverInfo": {
      "name": "mi-servidor-completo",
      "version": "1.0.0"
    }
  }
}
```

`tools/call`

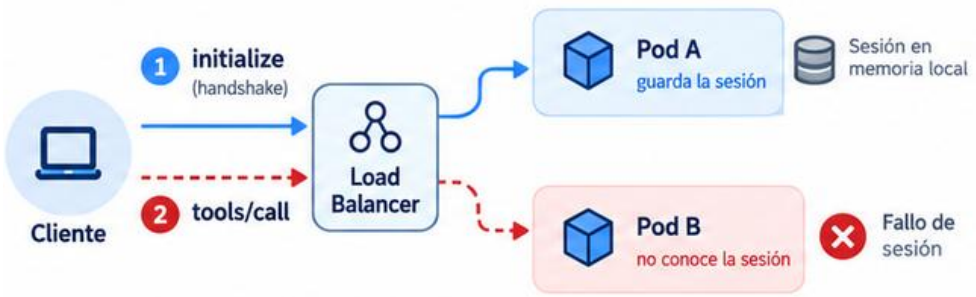
```json
{
  "jsonrpc": "2.0",
  "id": "msg-042",
  "method": "tools/call",
  "params": {
    "name": "sumar",
    "arguments": {
      "a": 5,
      "b": 10
    },
    "_meta": {
      "protocolVersion": "2026-07-28",
      "clientInfo": {
        "name": "Cursor",
        "version": "1.2.0"
      }
    }
  }
}
```

Comunicación MCP

¿Por qué eliminar el Handshake? Versión: 2026-07-28 → stateless

CON HANDSHAKE (con sesión)

Problema: cuello de botella



1. **Petición 1 (initialize)**: handshake al Pod A, que guarda la sesión
2. **Petición 2 (tools/call)**: el balanceador redirige la ejecución al Pod B.
3. **Fallo de sesión**: el Pod B no tiene constancia del handshake y rechaza la llamada.
4. **Solución parche**: sticky sessions o una DB centralizada.

Una empresa al desplegar su propio servidor MCP, **lo normal es ejecutarlo en su infraestructura, con varias réplicas detrás de un balanceador.** El modelo basado en handshake y sesión, complica estos despliegues a gran escala.

Por eso, hasta esta actualización, muchos servidores MCP se ejecutaban directamente en el ordenador del usuario.

Incluso integraciones de grandes proveedores seguían este modelo: el MCP corría en local y desde ahí se conectaba a las APIs externas.

Google Workspace (MCP Local)

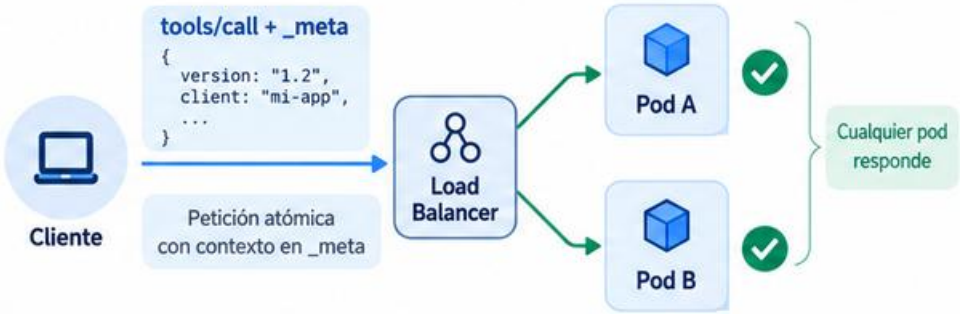
```
{
  "mcpServers": {
    "google-workspace": {
      "type": "stdio",
      "command": "gws",
      "args": ["mcp","-s","drive,gmail,calendar"]
    }
  }
}
```

Comunicación MCP

¿Por qué eliminar el Handshake? Versión: 2026-07-28 → stateless

SIN HANDSHAKE (stateless)

Solución: Peticiones auto-contenidas



1. **Sin dependencia de sesión:** desaparece el handshake.
2. **Contexto _meta**: Cada llamada tiene la versión y datos del cliente.
3. **Cualquier pod responde**: Cualquier réplica puede procesar la petición.

Ahora cada petición es auto-contenida. Ya no depende de un initialize previo ni de una sesión almacenada en un pod concreto. La versión del protocolo, la identidad del cliente y sus capacidades viajan con cada request, por lo que cualquier réplica puede atenderla.

Github (MCP Remoto)

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

Comunicación MCP

EBIS Business Techschool

Flujo de Comunicación



2. CICLO DE INTERCAMBIO

1. **Tool List**: El cliente solicita la lista de tools (`tools/list`) disponibles.
2. **Tool Call**: El cliente envía un método (`tools/call`) con la tool a ejecutar con sus `params` y un `id` único.
3. **Tool Result**: El servidor solicita y devuelve el result con el mismo `id`.

tools/list

```
{
  "jsonrpc": "2.0",
  "id": "msg-001",
  "result": {
    "tools": [
      {
        "name": "obtener_clima",
        "description": "Obtiene las condiciones meteorológicas actua
        "inputSchema": {
          "type": "object",
          "properties": {
            "ciudad": {
              "type": "string",
              "description": "El nombre de la ciudad, ej. Madrid, Bu
            },
            "unidades": {
              "type": "string",
              "enum": ["celsius", "fahrenheit"],
              "description": "Unidad de temperatura (por defecto cel
            }
          },
          "required": ["ciudad"]
        },
        "outputSchema": {
          "type": "object",
          "properties": {
            "temperatura": {
              "type": "string",
              "description": "La temperatura devuelta en formato tex
            }
          },
          "required": ["temperatura"],
          "additionalProperties": false
        }
      }
    ]
```

tools/call

```
{
  "jsonrpc": "2.0",
  "id": "msg-002",
  "method": "tools/call",
  "params": {
    "name": "obtener_clima",
    "arguments": {
      "ciudad": "Madrid",
      "unidades": "celsius"
    }
  }
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": "msg-002",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "{\"temperatura\": \"35.6\"}"
      }
    ],
    "isError": false
  }
}
```

Comunicación MCP

EBIS Business Techschool

Métodos disponibles: `General`

Método	Descripción		
`initialize`	El cliente lo envía al arrancar (versión, capacidades..)	⚠️	v2025-11-25
`notifications/initialized`	El cliente avisa al servidor que la sesión está activa	❌	
`ping`	Comprobar que la conexión sigue viva y el servidor responde	⚠️	
`server/discover`	Permite al cliente preguntarle al servidor qué versión habla, cómo se llama y qué capacidades tiene, de forma atómica y **sin abrir ninguna sesión**.	🆕	v2026-07-28
`notifications/progress`	Permite al servidor o al cliente reportar el avance porcentual o pasos de una tarea que tarda tiempo en completarse.	🆕	
`cancelled`	Se utiliza para abortar una petición		

Comunicación MCP

EBIS Business Techschool

Métodos disponibles: `Tools`

Método	Descripción
`tools/list`	El cliente solicita la lista de tools y sus schemas.
`tools/call`	El cliente solicita la invocación de una herramienta.
`notifications/tools/list_changed`	El servidor anuncia cambio en las tools

Métodos disponibles: `Resources`

Método	Descripción
`resources/list`	El cliente solicita la lista de tools y sus schemas.
`resources/templates/list`	Devuelve plantillas de URIs dinámicas (ej. `mcp://logs/{fecha}`)
`resources/read`	Solicitud de leer el contenido del recurso
`notifications/resources/list_changed`	El servidor anuncia cambio en las resources

```
› Lee mi base de datos en @mcp://proyecto/database_schema y dime como optimizarla
```

Métodos disponibles: `prompts`

Método	Descripción
`prompt/list`	Devuelve la lista de todas las plantillas de prompts disponibles y qué variables requieren
`prompts/get`	El cliente solicita el contenido de un prompt.

```
› /prompt_review_codigo_v5
```

Comunicación MCP

EBIS Business Techschool

`y mucho mas…`

Método	Descripción
`sampling/createMessage`	Intercambio de roles. , el servidor se convierte temporalmente en el "usuario" para pedirle razonamiento interno al modelo
`paginacion`	Permtir paginación para listar largas de elementos.

Comunicación MCP

EBIS Business Techschool

¿Cómo viaja el contenido entre cliente-servidor?

`stdio`: Comuniación local y privada donde dos programas en el mismo ordenador interactúan directamente entre ellos, sin usar internet ni puertos de red.

```
[mcpServers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]
enabled = true

[mcpServers.context7.env]
# Necesitarás generar tu propia API Key en el panel de control de Context7
CONTEXT7_API_KEY = "tu_api_key_aqui"
```

`HTTP (con SSE):` Un protocolo de comunicación por red (lo habitual cuando MCPs externos).

```
[mcp_servers.stitch]
url = "https://stitch.googleapis.com/mcp"

[mcp_servers.stitch.http_headers]
"X-Goog-Api-Key" = "XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
```

Arquitectura del Servidor MCP



EBIS Business Techschool

Arquitectura del Servidor MCP

EBIS Business Techschool

Responsabilidades del Servidor MCP

El servidor MCP no contiene la lógica de negocio; simplemente **es la capa estándar de adaptación**.



EXPOSICIÓN

Decide qué funcionalidades estarán disponibles (e.g., `create_user`).



DESCRIPCIÓN

Explica semánticamente qué hace cada función para el LLM.



DESCUBRIMIENTO

Permite al cliente preguntar por herramientas y recursos.



EJECUCIÓN

Transforma `tools/call` en llamadas reales al servicio (API/SQL).



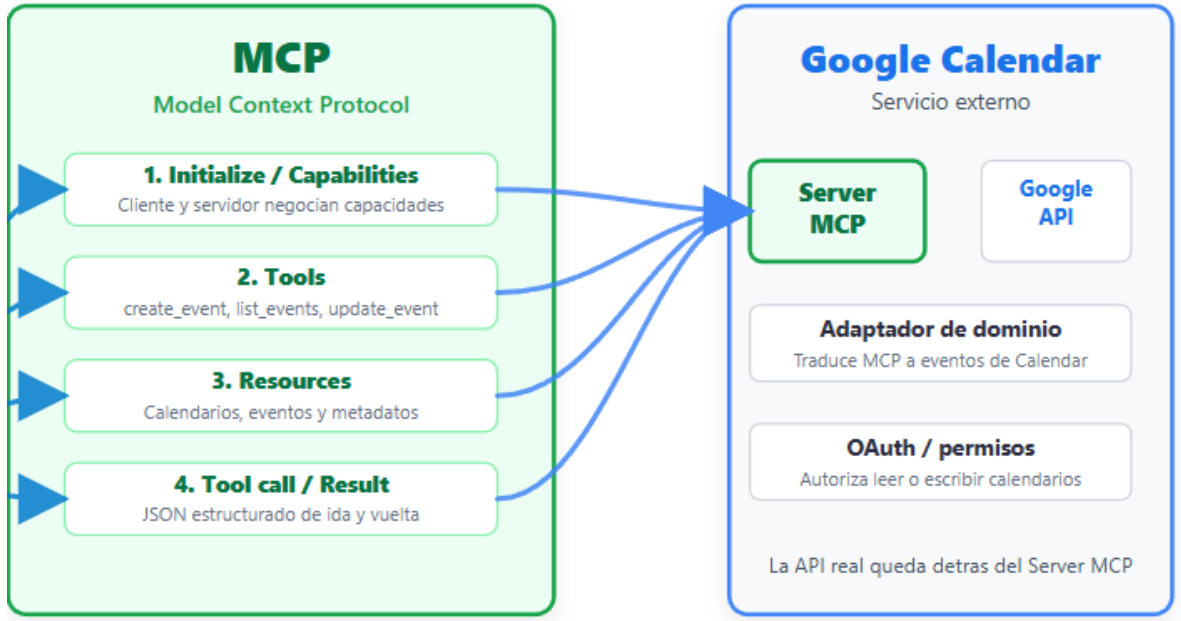
ESTANDARIZACIÓN

Convierte formatos propietarios al estándar MCP.

Arquitectura del Servidor MCP

EBIS Business Techschool

Responsabilidades del Servidor MCP



MCP no reemplaza tu API, construye una interfaz estándar para IA sobre ella.

Arquitectura del Servidor MCP

EBIS Business Techschool

Librería MCP en Python

La librería mcp es el SDK que usamos para crear servers MCP en Python sin tener que desarrollar todo el protocolo desde cero



Arquitectura del Servidor MCP

EBIS Business Techschool

Librería MCP en Python **Versión: 2025-11-25**

La librería nos da una clase principal para levantar un servidor de forma **muy sencilla**: `FastMCP`

Al igual que en FastAPI usamos decoradores para declarar **tool, resources y prompts**

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Demo MCP")


@mcp.tool()
def saludar(nombre: str) -> str:
    """Devuelve un saludo."""
    return f"Hola, {nombre}"


mcp.run()
```

STDIO

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP(
    "Demo MCP",
    host="127.0.0.1",
    port=8001,
    stateless_http=True,
    json_response=True,
)


@mcp.tool()
def saludar(nombre: str) -> str:
    """Devuelve un saludo."""
    return f"Hola, {nombre}"


mcp.run(transport="streamable-http")
```

HTTP

Arquitectura del Servidor MCP

EBIS Business Techschool

Librería MCP en Python

Versión: 2026-07-28 → stateless

En la nueva versión 2.0, **FastMCP** cambia de nombre a **MCPServer**.
Además, **la configuración del servidor** (host, puertos, etc…) se han movido **al método `run(…)`**.

El resto se mantiene igual.

```python
from mcp.server import MCPServer

mcp = MCPServer("Demo MCP")


@mcp.tool()
def saludar(nombre: str) -> str:
    """Devuelve un saludo."""
    return f"Hola, {nombre}"


mcp.run()
```

STDIO

```python
from mcp.server import MCPServer

mcp = MCPServer("Demo MCP")


@mcp.tool()
def saludar(nombre: str) -> str:
    """Devuelve un saludo."""
    return f"Hola, {nombre}"


mcp.run(
    transport="streamable-http",
    host="127.0.0.1",
    port=8001,
    stateless_http=True,
    json_response=True,
)
```

HTTP

Arquitectura del Servidor MCP

EBIS Business Techschool

Librería MCP en Python

`@mcp.tool()`

El decorador tiene 2 usos:

- **No se escribe JSON schema** a mano: la librería lo deduce de los parámetros tipos de la función.
- **Ejecuta dicha función** en caso de que la tool sea llamada (*la función realizará la llamada correspondiente al servidor real*)

```python
@mcp.tool()
def sumar(a: int, b: int) -> int:
    """Suma dos numeros."""
    return a + b
```

El decorador registra la función dentro del servidor MCP.



```json
{
  "name": "sumar",
  "description": "Suma dos numeros.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "a": {
        "type": "integer"
      },
      "b": {
        "type": "integer"
      }
    },
    "required": ["a", "b"]
  }
}
```

Schema Generado

Arquitectura del Servidor MCP

EBIS Business Techschool

Librería MCP en Python

`@mcp.tool()`

El decorador tiene 2 usos:

- **No se escribe JSON schema** a mano: la librería lo deduce de los parámetros tipos de la función.
- **Ejecuta dicha función** en caso de que la tool sea llamada (*la función realizará la llamada correspondiente al servidor real*)

```
› Usa la tool sumar para sumar 7 y 5.
```

Resource Response


```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "12"
      }
    ],
    "isError": false
  }
}
```

Arquitectura del Servidor MCP

EBIS Business Techschool

Librería MCP en Python

`@mcp.resource("<custom_uri>")`

El decorador tiene 2 usos:

- Declara un recurso que el cliente MCP puede leer usando una **URI**. Ejemplo: **`config://app`**
- Ejecuta la función decorada cuando ese recurso es consultado.

```python
@mcp.resource("config://app")
def get_app_config() -> str:
    """Devuelve la configuracion publica de la aplicacion."""
    return (
        "APP_NAME=Inventory Demo\n"
        "ENVIRONMENT=development\n"
        "API_URL=http://127.0.0.1:8000"
    )
```

Un ejemplo típico de resource:
Describir la config del servidor



```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "resources": [
      {
        "uri": "config://app",
        "name": "get_app_config",
        "description": "Devuelve la configuracion publica de la aplicacion.",
        "mimeType": "text/plain"
      }
    ]
  }
}
```

Schema Generado

Arquitectura del Servidor MCP

EBIS Business Techschool

Librería MCP en Python

`@mcp.resource("<custom_uri>")`

El decorador tiene 2 usos:

- Declara un recurso que el cliente MCP puede leer usando una **URI**. Ejemplo: `config:// app`
- Ejecuta la función decorada cuando ese recurso es consultado.

```
› Lee el recurso config://app del servidor MCP Demo Resources y dime qué configuración pública tiene la
  aplicación.
```




```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "contents": [
      {
        "uri": "config://app",
        "mimeType": "text/plain",
        "text": "APP_NAME=Inventory Demo\nENVIRONMENT=development\nAPI_URL=http
      }
    ]
  }
}
```


Resource Response

Arquitectura del Servidor MCP

EBIS Business Techschool

Librería MCP en Python

`@mcp.prompt()`

El decorador tiene 2 usos:

- Declara una plantilla de prompt que el cliente MCP puede reutilizar con parámetros.
- Ejecuta la función decorada cuando el prompt es solicitado.

```python
@mcp.prompt()
def crear_plan_estudio(tema: str, nivel: str) -> str:
    """Crea un prompt para generar un plan de estudio."""
    return (
        f"Quiero aprender sobre {tema}.\n"
        f"Mi nivel actual es {nivel}.\n"
        "Genera un plan de estudio de 5 pasos, con ejemplos practicos."
    )
```

Schema Generado

```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "prompts": [
      {
        "name": "crear_plan_estudio",
        "description": "Crea un prompt para generar un plan de estudio.",
        "arguments": [
          {
            "name": "tema",
            "description": "Argumento tema",
            "required": true
          },
          {
            "name": "nivel",
            "description": "Argumento nivel",
            "required": true
          }
        ]
```

Arquitectura del Servidor MCP

EBIS Business Techschool

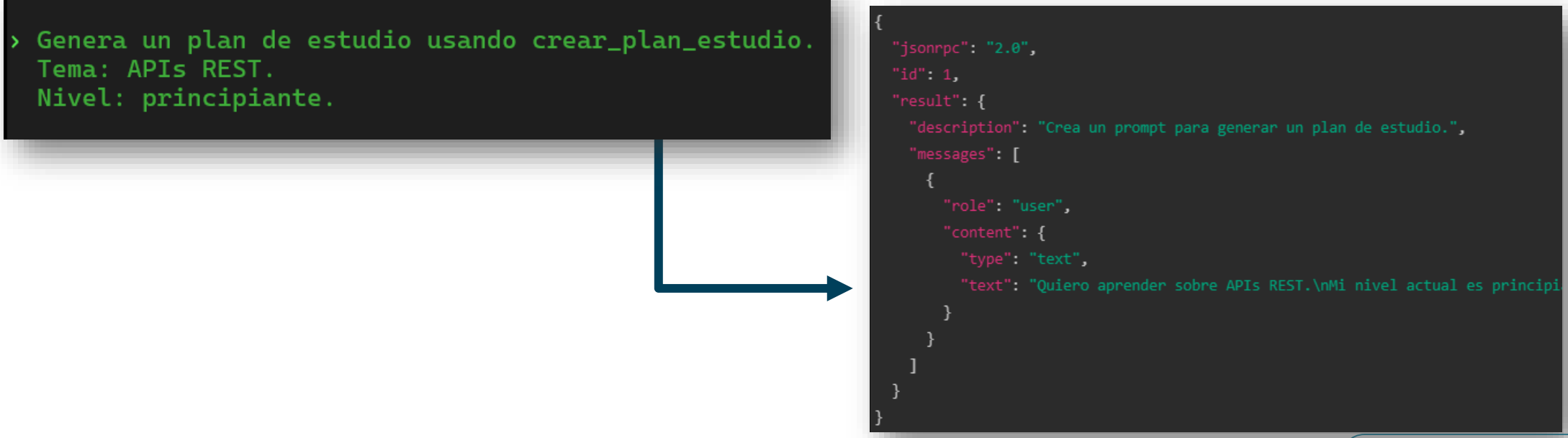
Librería MCP en Python

`@mcp.prompt()`

El decorador tiene 2 usos:

- Declara una plantilla de prompt que el cliente MCP puede reutilizar con parámetros.
- Ejecuta la función decorada cuando el prompt es solicitado.

```
› Genera un plan de estudio usando crear_plan_estudio.
  Tema: APIs REST.
  Nivel: principiante.
```



Añadir el servidor MCP a un CLI

EBIS Business Techschool

VSCode → `.vscode/mcp.json`

```
{
  "servers": {
    "miServer": {
      "type": "stdio",
      "command": "uv",
      "args": [
        "--directory",
        "C:\\ruta\\a\\mi-server-mcp",
        "run",
        "python",
        "main.py"
      ],
      "env": {
        "API_URL": "http://127.0.0.1:8000",
        "API_TOKEN": "Bearer mi-token"
      }
    }
  }
}
```

Stdio

```
{
  "inputs": [
    {
      "type": "promptString",
      "id": "mi-server-mcp-token",
      "description": "Token del MCP server",
      "password": true
    }
  ],
  "servers": {
    "miServer": {
      "type": "http",
      "url": "http://127.0.0.1:8001/mcp",
      "headers": {
        "Authorization": "Bearer ${input:mi-server-mcp-token}"
      }
    }
  }
}
```

HTTP

Añadir el servidor MCP a un CLI

EBIS Business Techschool

Codex → `.codex\config.toml`

```toml
[mcp_servers.mi-server]
command = "uv"
args = [
  "--directory",
  "C:\\ruta\\a\\mi-server-mcp",
  "run",
  "python",
  "main.py",
]

[mcp_servers.mi-server.env]
SERVICE_API_URL = "http://127.0.0.1:8000"
MCP_SERVER_API_TOKEN = "Bearer mi-token"
```

Stdio

```toml
[mcp_servers.mi-server]
url = "http://127.0.0.1:8001/mcp"
bearer_token_env_var = "MI_SERVER_MCP_TOKEN"


# POSTERIORMENTE SE DEBE TENER UNA ENV CON
# EL MISMO NOMBRE, EJEMPLO:
# export MI_SERVER_MCP_TOKEN = "mi-token-mcp"
```

HTTP

MCP Inspector



EBIS Business Techschool

¿Qué es MCP Inspector?

EBIS Business Techschool

Un cliente MCP de laboratorio: prueba servidores reales sin montar un agente ni escribir tu propio cliente.

Conectar

Abre una sesión con un servidor usando el transporte MCP elegido.

Descubrir

Muestra las capacidades publicadas: tools, resources y sus esquemas.

Probar y observar

Ejecuta llamadas y permite inspeccionar los mensajes entre cliente y servidor.

Idea clave: Inspector es una interfaz visual para actuar como cliente MCP.

La demo: Inspector → servidor MCP → API

EBIS Business Techschool

Tres procesos con responsabilidades distintas; cada flecha usa su propio protocolo.

MCP INSPECTOR

Cliente de pruebas
con interfaz web

Streamable HTTP →

EBIS CALENDAR

Servidor MCP
traduce tools a REST

HTTP →

CALENDAR API

API REST
127.0.0.1:8000

MCP estandariza el acceso

El Inspector descubre y llama tools con mensajes MCP.

La API conserva la lógica

El servidor MCP actúa como adaptador; no reemplaza la API.

01 · Arranca la API de calendario

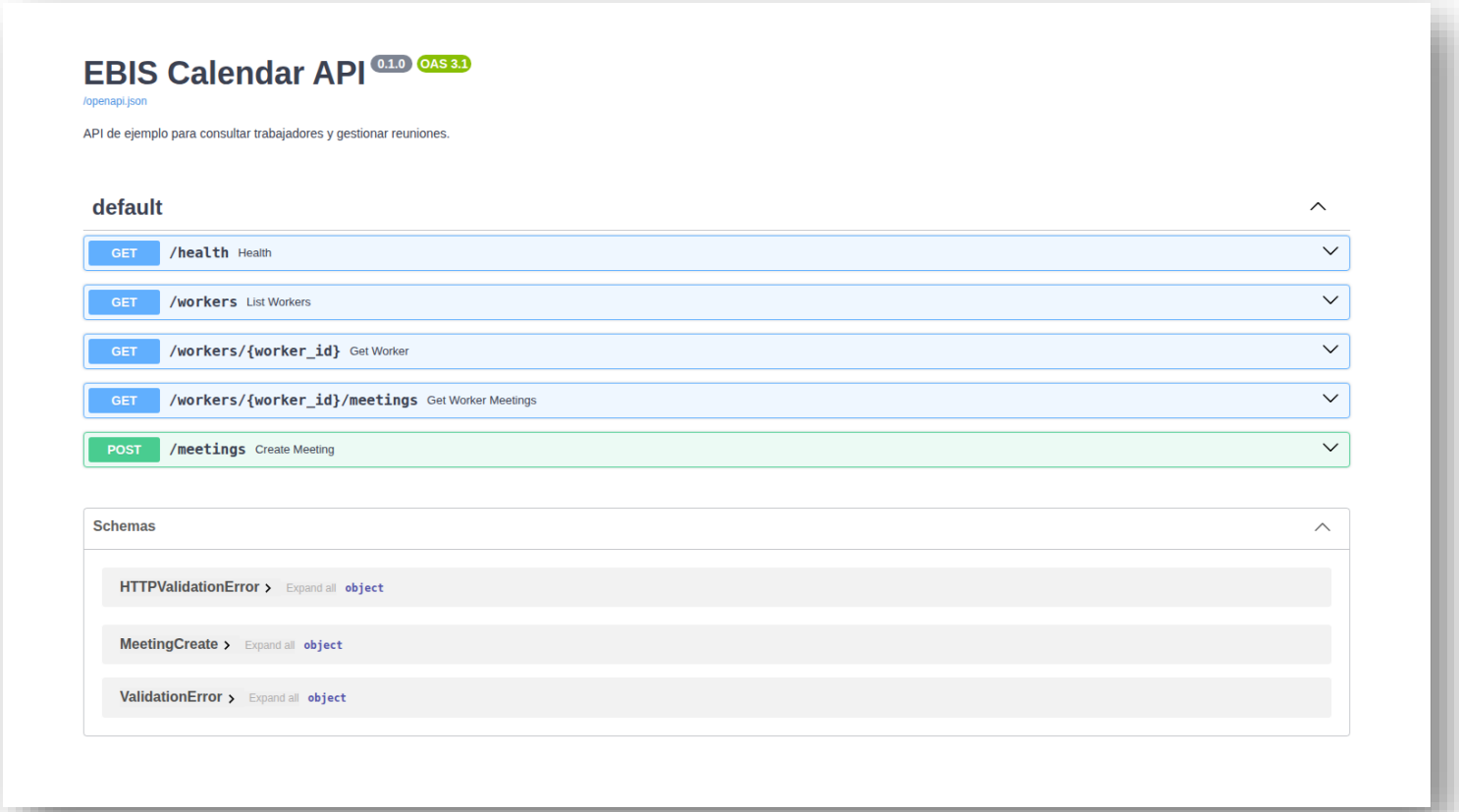
EBIS Business Techschool

TERMINAL 1

```
cd ./ebis-calendar-api
uv sync
uv run python main.py

# En otra terminal:
curl --fail --silent --show-error \
  http://127.0.0.1:8000/health

{"status":"ok"}
```



La API REST escucha en :8000 · documentación: /docs

02 · Arranca el servidor MCP

EBIS Business Techschool

TERMINAL 2

```
cd ./ebis-calendar-api-mcp

uv sync

export MCP_TRANSPORT=http

uv run python -m ebis_calendar_api_mcp.server
```

ENDPOINT MCP

http://127.0.0.1:8001/mcp

Streamable HTTP

Deja esta terminal y la de la API abiertas. El servidor MCP llamará a la API REST.

Si arranca correctamente, escuchará en 127.0.0.1:8001/mcp.

03 · Conecta el Inspector oficial y MCP Server via HTTP

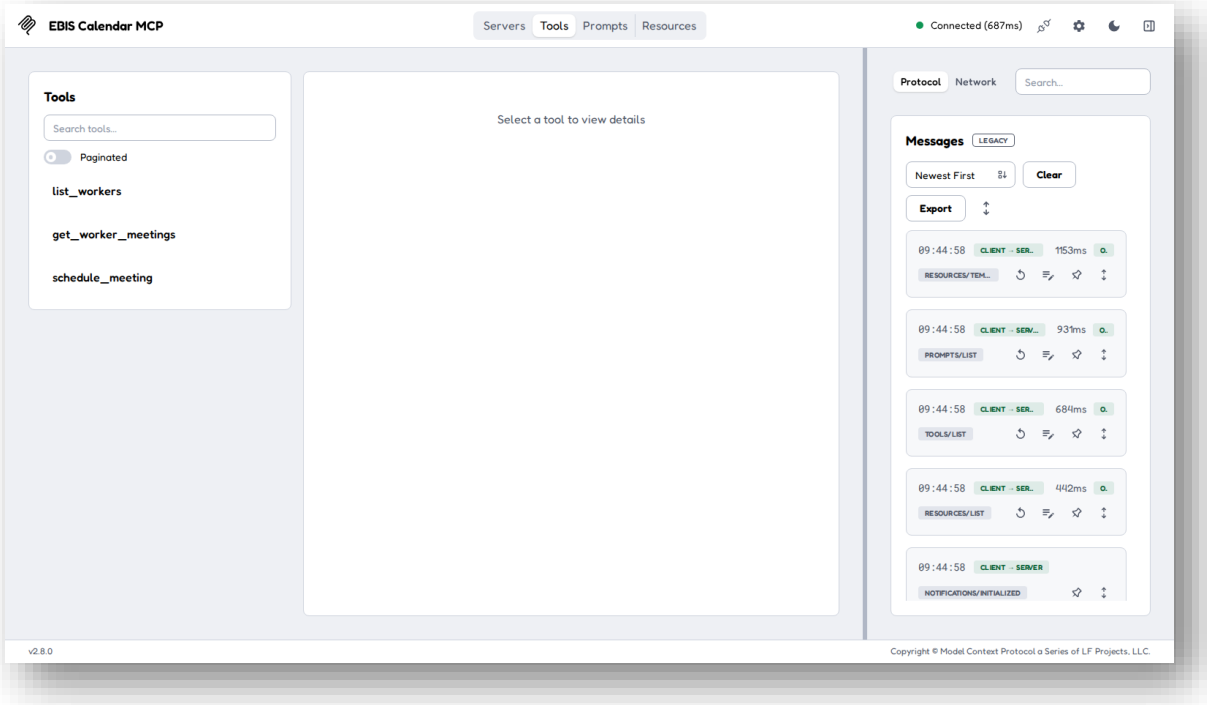
EBIS Business Techschool

Requisito: Node.js 22.19+ y npx

TERMINAL 3

```
npx -y @modelcontextprotocol/inspector \
  --protocol-era legacy \
  --server-url http://127.0.0.1:8001/mcp \
  --transport http
```

1. Abre en el navegador la URL local que imprime la terminal.
2. Conecta y confirma el estado Connected.
3. No compartas la URL: contiene un token temporal.
4. Con **`--protocol-era legacy`** permite conectarse a servidores con versiones anteriores a la 28 de Julio de 2027.



La pestaña Tools muestra lo que publica el servidor.

03 · Conecta el Inspector oficial y MCP Server via HTTP

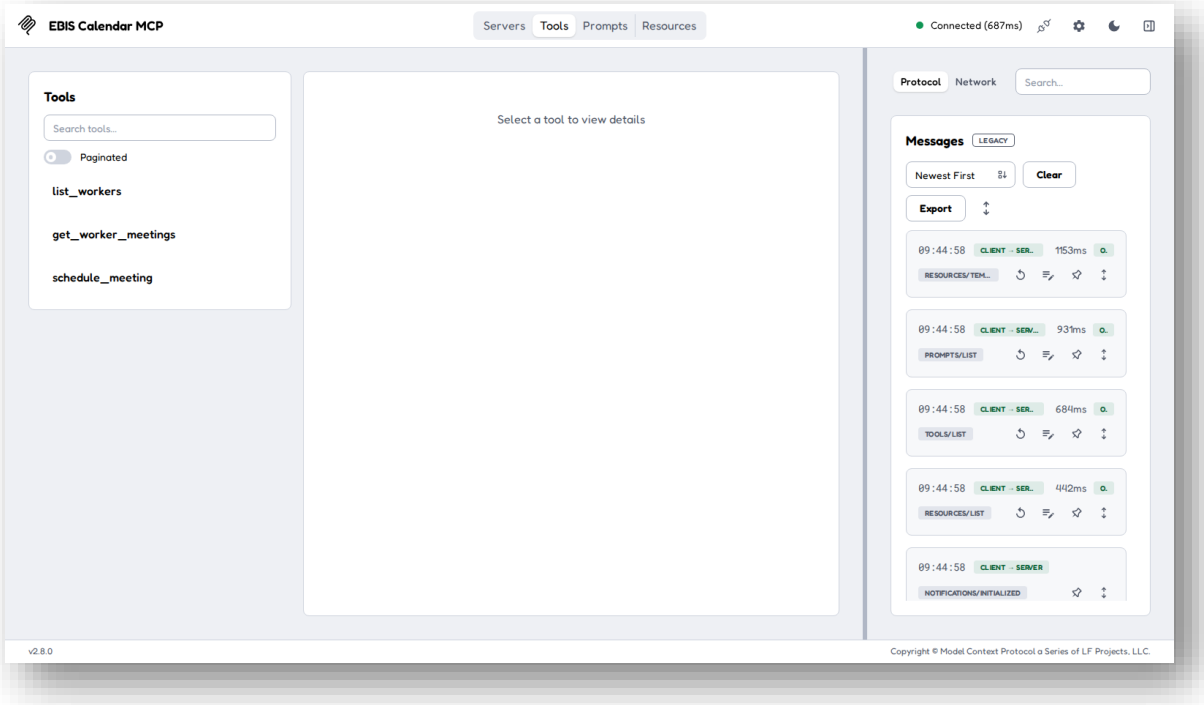
EBIS Business Techschool

Requisito: Node.js 22.19+ y npx

TERMINAL 3

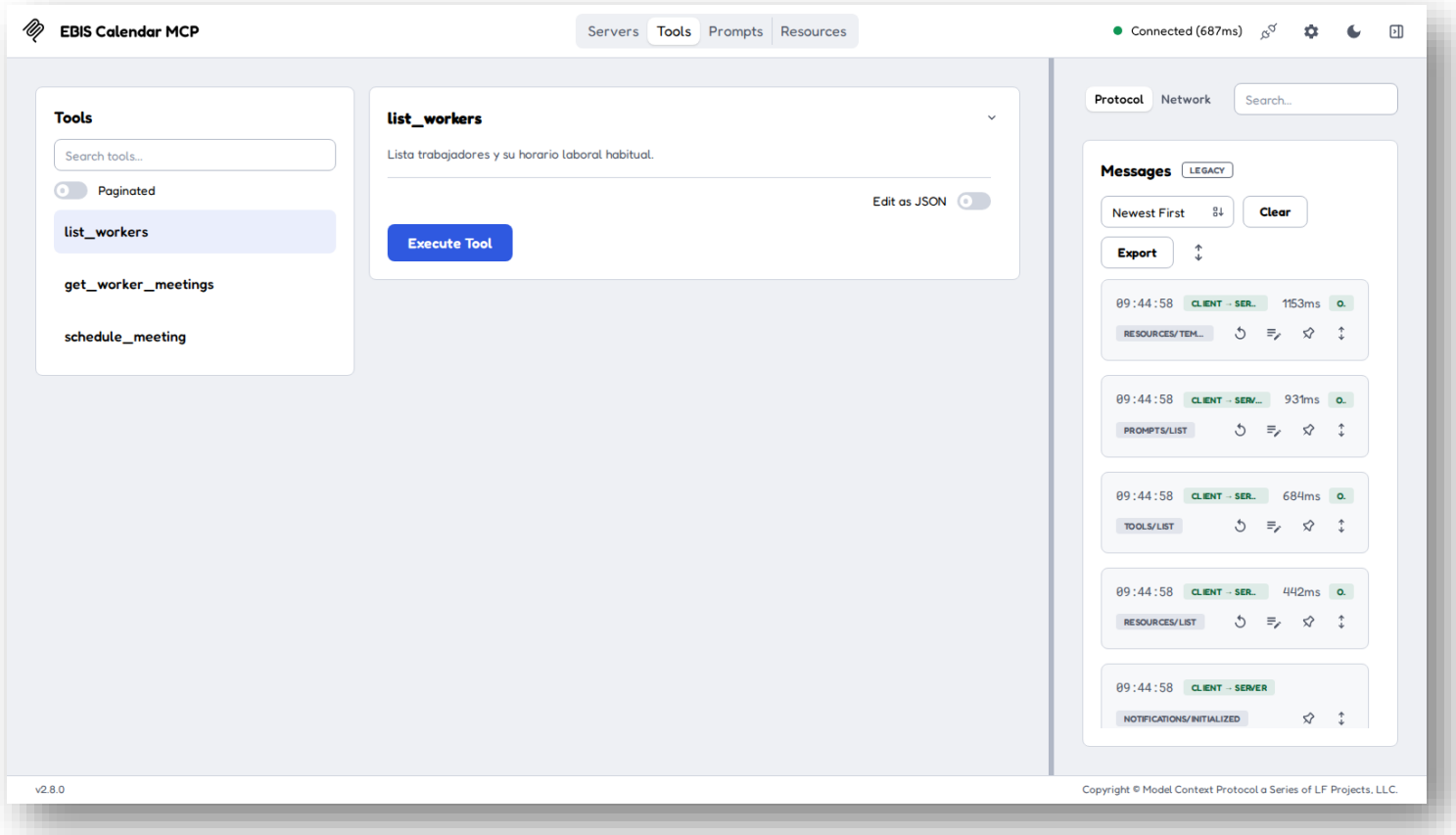
```
npx -y @modelcontextprotocol/inspector \
  --protocol-era auto \
  --server-url http://127.0.0.1:8001/mcp \
  --transport http
```

1. Lo mismo que antes
4. Con **`--protocol-era auto`** utiliza el nuevo protocolo MCP 2.0 (sin handshake)



La pestaña Tools muestra lo que publica el servidor.

04 · Descubre tools y revisa sus esquemas



Selecciona una tool para ver su descripción y el formulario.

TOOLS PUBLICADAS

list_workers

Sin argumentos

get_worker_meetings

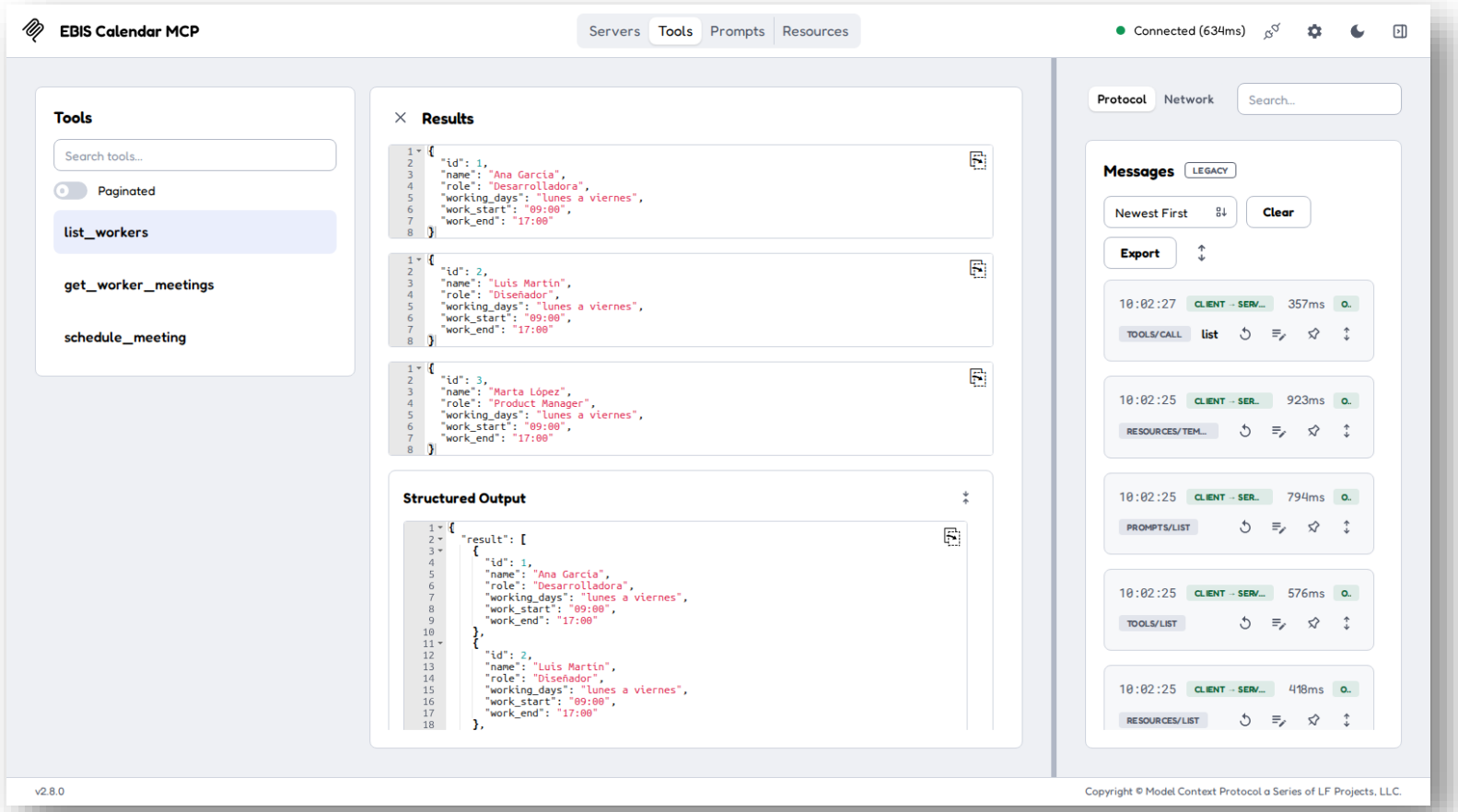
worker_id requerido · day opcional (AAAA-MM-DD)

schedule_meeting

worker_id, title, start y end

05 · Ejecuta tools/call y lee el resultado

EBIS Business Techschool



La respuesta incluye los datos devueltos por la API.

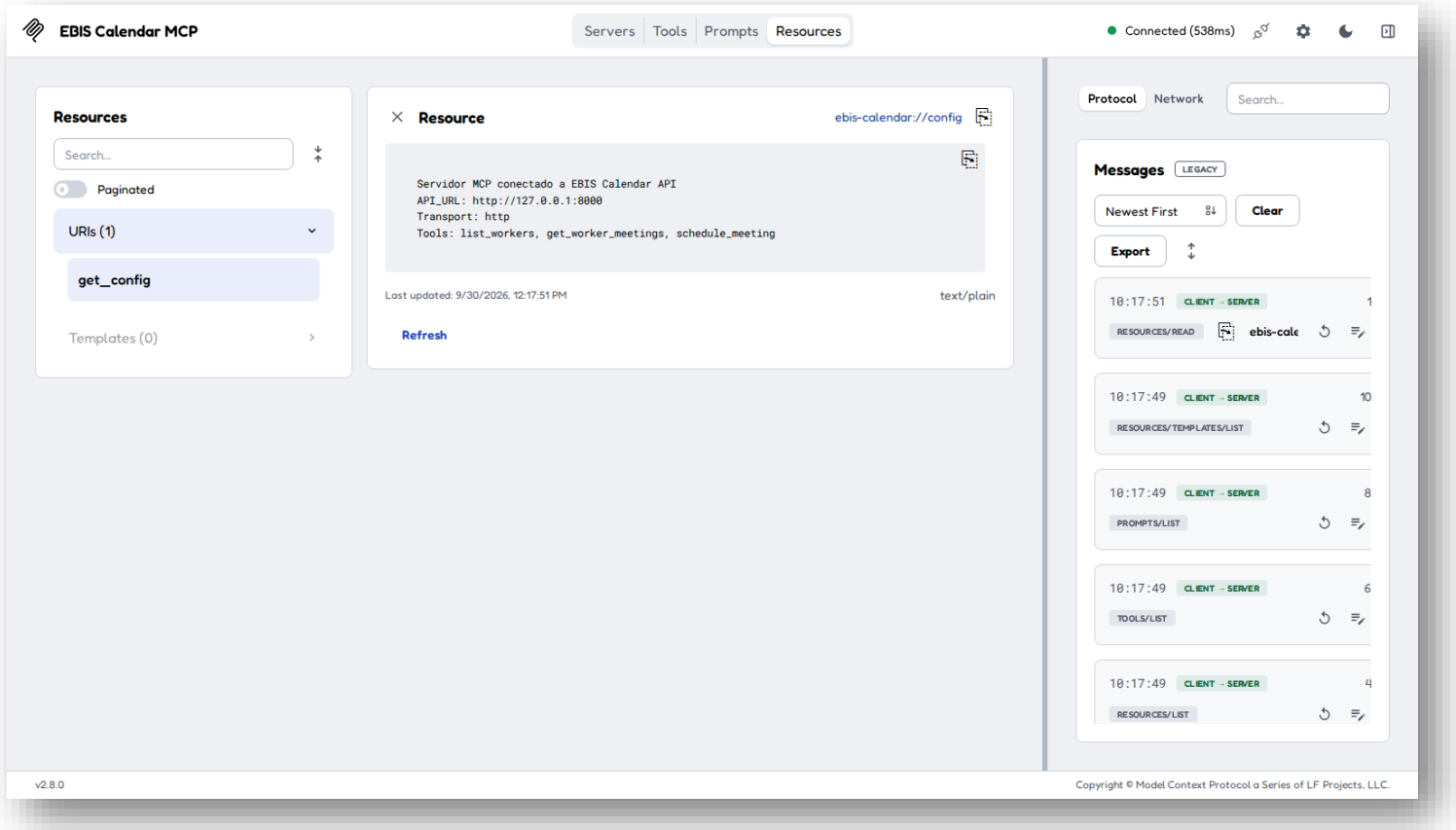
CICLO DE LA LLAMADA

1. **Pulsas Execute Tool**
2. Inspector envía tools/call
3. El servidor consulta la API REST
4. **Inspector muestra la respuesta**

Ana · Luis · Marta

06 · Lee un resource

EBIS Business Techschool



Tool = acción

Se invoca para realizar una operación. Ej.: list_workers.

Resource = información

Se lee desde una URI. Ej.: ebis-calendar://config.

En Resources, selecciona get_config: Inspector lee y muestra su contenido.

07 · Prueba una tool con argumentos

EBIS Business Techschool

get_worker_meetings

Consulta de solo lectura

```
worker_id: 1
day: "2026-10-01"  # opcional

day usa el formato AAAA-MM-DD.
```

schedule_meeting

Crea una reunión en la API

```
worker_id: 1
title: "Reunión de demo"
start: "2026-10-01T14:00:00"
end:   "2026-10-01T14:30:00"
```

Efecto real: elige un hueco futuro entre semana (09:00–17:00) y sin solapamientos. Los datos viven en memoria y se pierden al parar la API.

Transporte alternativo: stdio

EBIS Business Techschool

HTTP · proceso independiente

Arranca el servidor MCP en su terminal y conecta Inspector a su URL.

stdio · proceso gestionado

Inspector inicia el proceso MCP y se comunica por entrada/salida estándar.

Comando para probar la variante stdio.

TERMINAL · INICIO DIRECTO DE STDIO

```
npx -y @modelcontextprotocol/inspector \
  uv --directory ./ebis-calendar-api-mcp \
  run python -m ebis_calendar_api_mcp.server
```

El comando anterior utiliza la versión antigua de MCP

Transporte alternativo: stdio

EBIS Business Techschool

HTTP · proceso independiente

Arranca el servidor MCP en su terminal y conecta Inspector a su URL.

stdio · proceso gestionado

Inspector inicia el proceso MCP y se comunica por entrada/salida estándar.

Comando para probar la variante stdio.

TERMINAL · INICIO DIRECTO DE STDIO

```
npx -y @modelcontextprotocol/inspector \
  --protocol-era auto \
  uv --directory ./ebis-calendar-api-mcp \
  run python -m ebis_calendar_api_mcp.server
```

Con protocolo en modo AUTO se utiliza la versión MCP 2.0

Arquitectura del Cliente MCP



EBIS Business Techschool

Arquitectura del Cliente MCP

EBIS Business Techschool



HOST

Es la **aplicación principal** (Codex, VSCode, Cursor o tu CLI). Gestiona la interfaz de usuario y **el bucle agéntico**.

Es el responsable de levantar un Cliente MCP por cada MCP que exista configurado



CLIENTE MCP

Es la **una componente del HOST** que permite levantar una instancia MCP que mantiene una conexión 1:1 con un servidor MCP.
Si configuras en Codex 5 MCPs, se iniciarán 5 clientes MCP dentro de él.

Arquitectura del Cliente MCP

EBIS Business Techschool

También para crear MCP Clients



Arquitectura del Cliente MCP

EBIS Business Techschool

Levantando un Cliente MCP por STDIO

Versión: 2025-11-25

```python
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(
        command="uv",
        args=[
            "--directory",
            "C:/ruta/a/mi-server-mcp",
            "run",
            "python",
            "main.py",
        ],
        env={
            "API_URL": "http://127.0.0.1:8000",
            "API_TOKEN": "Bearer mi-token",
        },
    )
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools = await session.list_tools()
            print(tools)

asyncio.run(main())
```

Aquí Codex lee la config de los MCPs en config.toml. StdioServerParameters es un molde que guarda la config para ejecutar el Server MCP por STDIO

Inicia el proceso del servidor MCP
(recuerda que un Server MCP por STDIO es el Host quien lo "arranca")

abre una sesion MCP (para la comunicación)

Inicia la comunicación (handshake) y pide el listado de tools

Arquitectura del Cliente MCP

EBIS Business Techschool

Levantando un Cliente MCP por STDIO

Versión: 2026-07-28

```python
from mcp import Client, StdioServerParameters

async def main():
    server_params = StdioServerParameters(
        command="uv",
        args=[
            "--directory",
            "C:/ruta/a/mi-server-mcp",
            "run",
            "python",
            "main.py",
        ],
        env={
            "API_URL": "http://127.0.0.1:8000",
            "API_TOKEN": "Bearer mi-token",
        },
    )

    async with Client(server_params) as client:
        tools = await client.list_tools()
        print(tools)
```

Aquí Codex lee la config de los MCPs en config.toml. StdioServerParameters es un molde que guarda la config para ejecutar el Server MCP por STDIO

El nuevo cliente Client gestiona internamente el transporte y la conexión con el servidor MCP.

YA NO HAY HANDSHAKE. Se pide directamente el listado de tools

Arquitectura del Cliente MCP

EBIS Business Techschool

Levantando un Cliente MCP por HTTP

Versión: 2025-11-25

```python
import asyncio

from mcp import ClientSession
from mcp.client.streamable_http import streamable_http_client


async def main():
    url = "http://127.0.0.1:8001/mcp"

    async with streamable_http_client(url) as (read, write, _):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            print(tools)


asyncio.run(main())
```

Aquí Codex lee la config de los MCPs en config.toml

Se conecta a un MCP server que ya existe. *No arranca nada.*

Posteriormente, de la misma forma: se abre una sesión, se inicia la comunicación y se piden tools.

Arquitectura del Cliente MCP

EBIS Business Techschool

Levantando un Cliente MCP por HTTP

Versión: 2026-07-28

```python
from mcp import Client


async def main():
    url = "http://127.0.0.1:8001/mcp"

    async with Client(url) as client:
        tools = await client.list_tools()
        print(tools)
```

Aquí Codex lee la config de los MCPs en config.toml

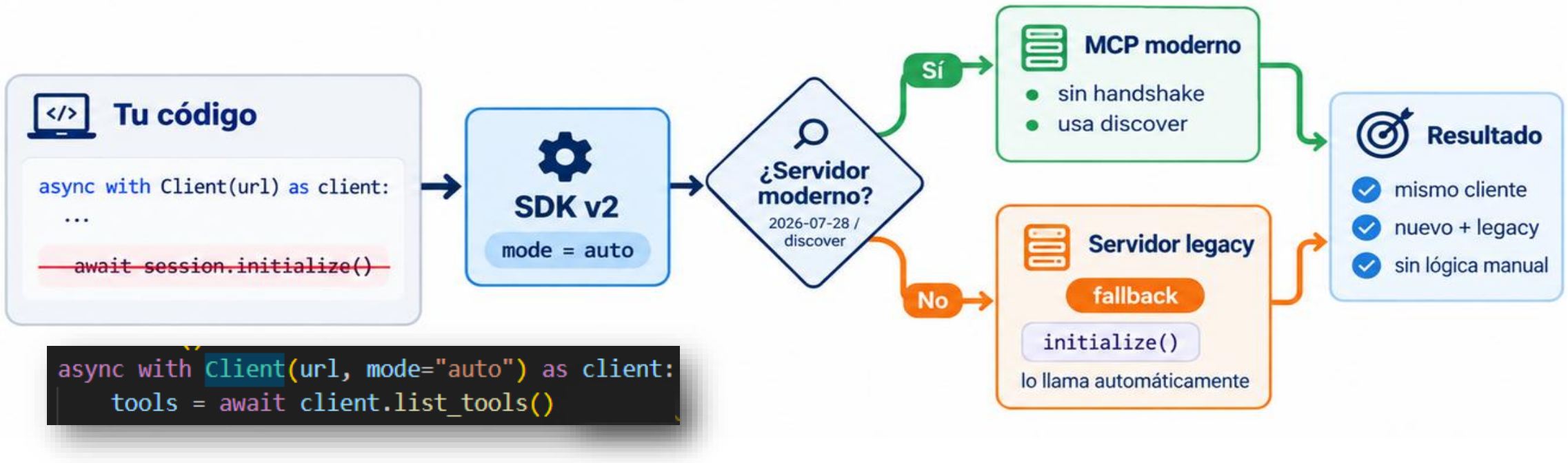
El nuevo cliente Client gestiona internamente el transporte y la conexión con el servidor MCP.

Posteriormente, el cliente está listo para hacer peticiones, por ejemplo, pedir tools. No requiere de HANDSHAKE

Arquitectura del Cliente MCP

EBIS Business Techschool

¿Qué ocurre si un servidor NO se ha actualizado a MCP 2.0?



Ahora el nuevo cliente incorpora un modo automático (**`mode="auto"`**) que detecta si el servidor usa MCP moderno y, si es legacy, hace por sí solo el fallback y ejecuta initialize().

Arquitectura del Cliente MCP

EBIS Business Techschool

Configuración de MCP Servers

VSCode → `.vscode/mcp.json`

```json
{
  "servers": {
    "miServer": {
      "type": "stdio",
      "command": "uv",
      "args": [
        "--directory",
        "C:\\ruta\\a\\mi-server-mcp",
        "run",
        "python",
        "main.py"
      ],
      "env": {
        "API_URL": "http://127.0.0.1:8000",
        "API_TOKEN": "Bearer mi-token"
      }
    }
  }
}
```

Stdio

```json
{
  "inputs": [
    {
      "type": "promptString",
      "id": "mi-server-mcp-token",
      "description": "Token del MCP server",
      "password": true
    }
  ],
  "servers": {
    "miServer": {
      "type": "http",
      "url": "http://127.0.0.1:8001/mcp",
      "headers": {
        "Authorization": "Bearer ${input:mi-server-mcp-token}"
      }
    }
  }
}
```

HTTP

Codex → `.codex\config.toml`

```toml
[mcp_servers.mi-server]
command = "uv"
args = [
  "--directory",
  "C:\\ruta\\a\\mi-server-mcp",
  "run",
  "python",
  "main.py",
]

[mcp_servers.mi-server.env]
SERVICE_API_URL = "http://127.0.0.1:8000"
MCP_SERVER_API_TOKEN = "Bearer mi-token"
```

Stdio

```toml
[mcp_servers.mi-server]
url = "http://127.0.0.1:8001/mcp"
bearer_token_env_var = "MI_SERVER_MCP_TOKEN"


# POSTERIORMENTE SE DEBE TENER UNA ENV CON
# EL MISMO NOMBRE, EJEMPLO:
# export MI_SERVER_MCP_TOKEN = "mi-token-mcp"
```

HTTP

En una app real (Codex, VSCode) **no se fijan en el código** la configuración de los MCP servers de forma estática.

Se leen desde un archivo de configuración. Y la app tiene la capacidad de levantar múltiples a la vez.

Arquitectura del Cliente MCP

EBIS Business Techschool

Configuración de MCP Servers

VSCode → `.vscode/mcp.json`

```
{
  "servers": {
    "miServer": {
      "type": "stdio",
      "command": "uv",
      "args": [
        "--directory",
        "C:\\ruta\\a\\mi-server-mcp",
        "run",
        "python",
        "main.py"
      ],
      "env": {
        "API_URL": "http://127.0.0.1:8000",
        "API_TOKEN": "Bearer mi-token"
      }
    }
  }
}
```

Stdio

```
{
  "inputs": [
    {
      "type": "promptString",
      "id": "mi-server-mcp-token",
      "description": "Token del MCP server",
      "password": true
    }
  ],
  "servers": {
    "miServer": {
      "type": "http",
      "url": "http://127.0.0.1:8001/mcp",
      "headers": {
        "Authorization": "Bearer ${input:mi-server-mcp-token}"
      }
    }
  }
}
```

HTTP

Codex → `.codex\config.toml`

```
[mcp_servers.mi-server]
command = "uv"
args = [
  "--directory",
  "C:\\ruta\\a\\mi-server-mcp",
  "run",
  "python",
  "main.py",
]

[mcp_servers.mi-server.env]
SERVICE_API_URL = "http://127.0.0.1:8000"
MCP_SERVER_API_TOKEN = "Bearer mi-token"
```

Stdio

```
[mcp_servers.mi-server]
url = "http://127.0.0.1:8001/mcp"
bearer_token_env_var = "MI_SERVER_MCP_TOKEN"


# POSTERIORMENTE SE DEBE TENER UNA ENV CON
# EL MISMO NOMBRE, EJEMPLO:
# export MI_SERVER_MCP_TOKEN = "mi-token-mcp"
```

HTTP

El Host lee la configuración y por cada server:

1. **Crea** un cliente MCP
2. **Abre** una sesion
3. **Descubre** tools/resources/prompts
4. **Mantiene** esa conexion durante la conversacion

Arquitectura del Cliente MCP

EBIS Business Techschool

Ejecuta una tool concreta del servidor MCP.

```
tools = await session.list_tools()
```

```
ListToolsResult(
    tools=[
        Tool(
            name="sumar",
            description="Suma dos numeros.",
            inputSchema={
                "type": "object",
                "properties": {
                    "a": {
                        "type": "integer"
                    },
                    "b": {
                        "type": "integer"
                    }
                },
                "required": ["a", "b"]
            }
        )
    ]
)
```



```
result = await session.call_tool(
    "sumar",
    arguments={"a": 2, "b": 3},
)
```

En un Agente, **el LLM es quien genera el diccionario de argumentos** gracias a haberle pasado **el schema de la tool.**

Arquitectura del Cliente MCP

EBIS Business Techschool

```python
messages = [
    {"role": "system", "content": "Eres un agente con tools locales."},
    {"role": "user", "content": user_goal},
]

tools = get_local_tool_schemas()

for _step in range(5):
    response = llm.chat(
        messages=messages,
        tools=tools,
    )

    message = response.message
    tool_calls = message.tool_calls

    if not tool_calls:
        print(message.content)
        break

    messages.append({
        "role": "assistant",
        "content": message.content,
    })
    for tool_call in tool_calls:
        result = execute_local_tool(
            name=tool_call.function.name,
            arguments=tool_call.function.arguments,
        )
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": result,
        })
```

AGENTE SIN MCP

El Host define sus propias **tools locales**.

El LLM solo puede usar las tools que la app haya programado.

Arquitectura del Cliente MCP

AGENTE CON MCP

El Host define sus propias **tools locales**.

El Host **descubre tools desde servidores MCP**.

El LLM puede usar tools que viven dentro y fuera de la app.

```python
mcp_clients = [
    MCPClient(name="math"),
]
local_tools = get_local_tool_schemas()

mcp_tools = []
for mcp_client in mcp_clients:
    discovered_tools = await mcp_client.list_tools()
    mcp_tools.extend(discovered_tools)

tools = local_tools + mcp_tools
for _step in range(5):
    response = llm.chat(...)

    message = response.message
    tool_calls = message.tool_calls
    if not tool_calls:
        print(message.content)
        break

    messages.append({
        "role": "assistant",
        "content": message.content,
    })
    for tool_call in tool_calls:
        tool_name = tool_call.function.name
        arguments = tool_call.function.arguments

        if is_mcp_tool(tool_name):
            result = await call_mcp_tool(
                clients=mcp_clients,
                name=tool_name,
                arguments=arguments,
            )
        else:
            result = execute_local_tool(...)
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": result,
```

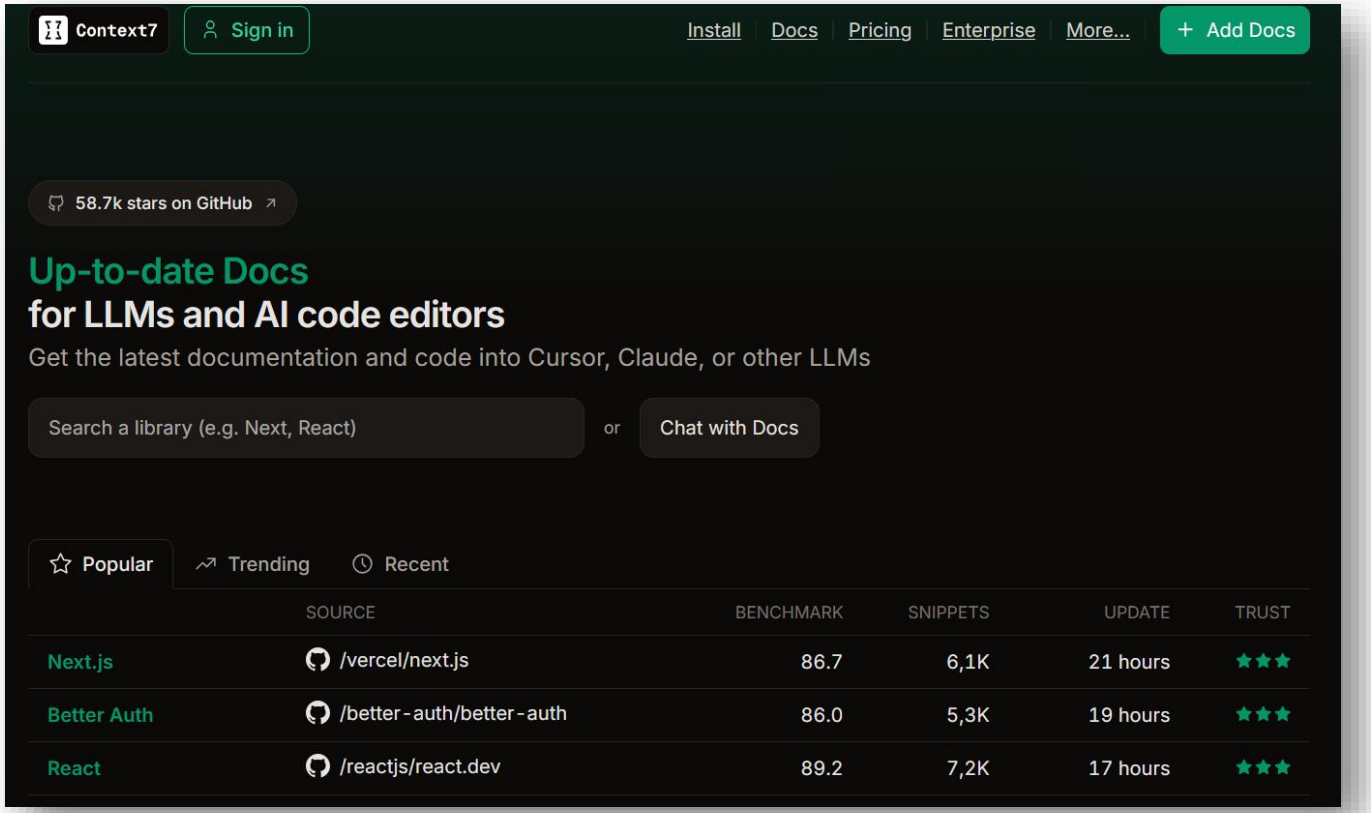
MCPs y otras estrategias



EBIS Business Techschool

MCPs Recomendados

EBIS Business Techschool

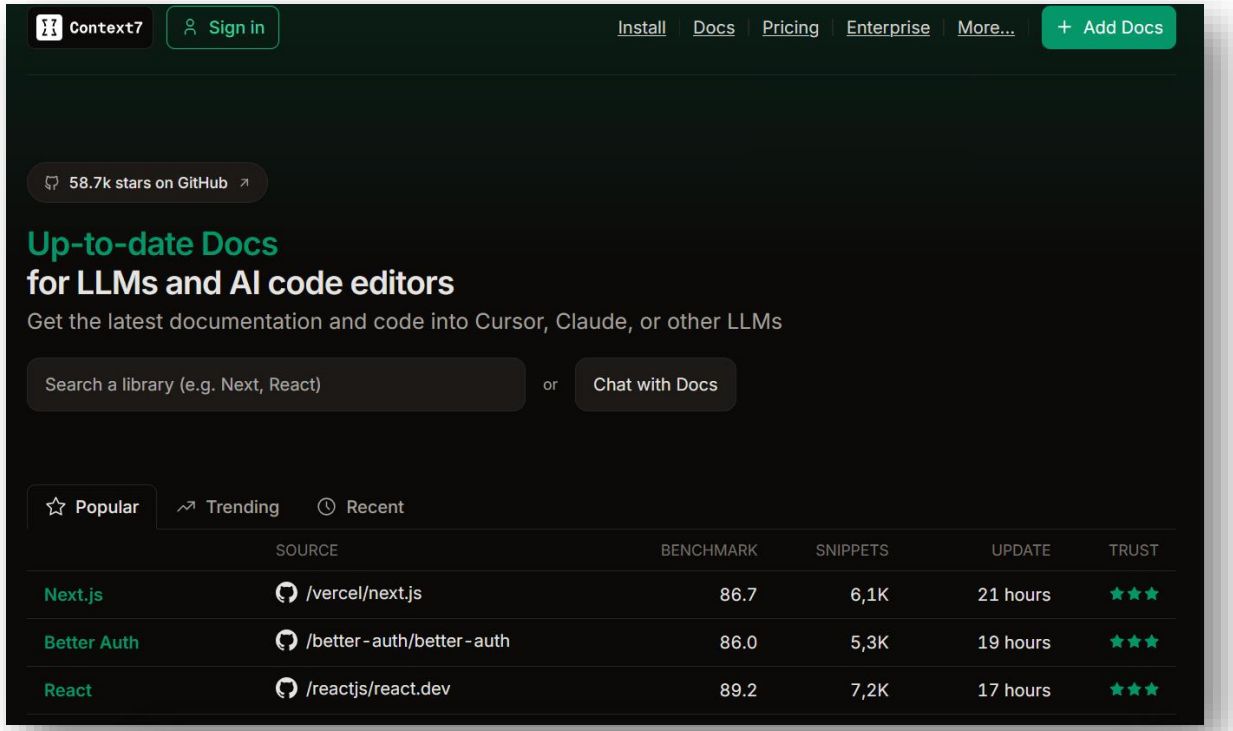


Context7 es un servidor MCP que **inyecta documentación oficial** y **actualizada** directamente **en el contexto de tu asistente de IA**, eliminando alucinaciones y el uso de código obsoleto.

MCPs Recomendados

Para configurarlo, **se añade el MCP Server en la configuración del Agente.**

En modo STDIO (se define el comando de para establecer la conexión local)




```
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]

[mcp_servers.context7.env]
CONTEXT7_API_KEY = "XXXXXXXXXXXXXXXXXXXXXXXXXX"
```

MCPs Recomendados

EBIS Business Techschool

```
› Podrías darme el changelog de la última versión de la librería de FastAPI. Y describeme en detalle el decorador @app.<verbo>

• Called
  └ context7.query-docs({"libraryId":"/tiangolo/fastapi","query":"What is the latest FastAPI release changelog
      and how do @app.get, @app.post and other HTTP method decorators work?"})
    Library /tiangolo/fastapi has been redirected to this library: /fastapi/fastapi.

─────────────────────────────────────────────────────────────────────────────────

• Context7 redirige la librería al identificador oficial /fastapi/fastapi; voy a consultar ahí y, para la
  versión exacta, revisaré también las releases oficiales.

      documentation for path operation decorators @app.get @app.post @app.put @app.delete @app.patch
      @app.options @app.head @app.trace."})
    Source: https://github.com/fastapi/fastapi/blob/master/docs/en/docs/tutorial/first-steps.md

    ```python

    ### Recap, step by step > Step 3: create a path operation > Define a path operation decorator

    Source: https://github.com/fastapi/fastapi/blob/master/docs/en/docs/tutorial/first-steps.md

    A path operation decorator, such as `@app.get('/')`, inform...

• Searching the web

• A fecha de hoy, 7 de julio de 2026, la última release publicada de FastAPI es 0.139.0, publicada el 1 de julio de 2026. GitHub la marca como
  Latest, y las release notes oficiales la listan como 0.139.0 (2026-07-01). Fuentes: GitHub Releases
  (https://github.com/fastapi/fastapi/releases) y FastAPI Release Notes (https://fastapi.tiangolo.com/release-notes/).

  Changelog FastAPI 0.139.0

  Cambio funcional principal:

  - Añadido soporte para dependencias en app.frontend(), por ejemplo para aplicar autenticación automática basada en cookies al frontend. PR
    #15908 (https://github.com/fastapi/fastapi/pull/15908).

  Cambios de documentación/traducciones:

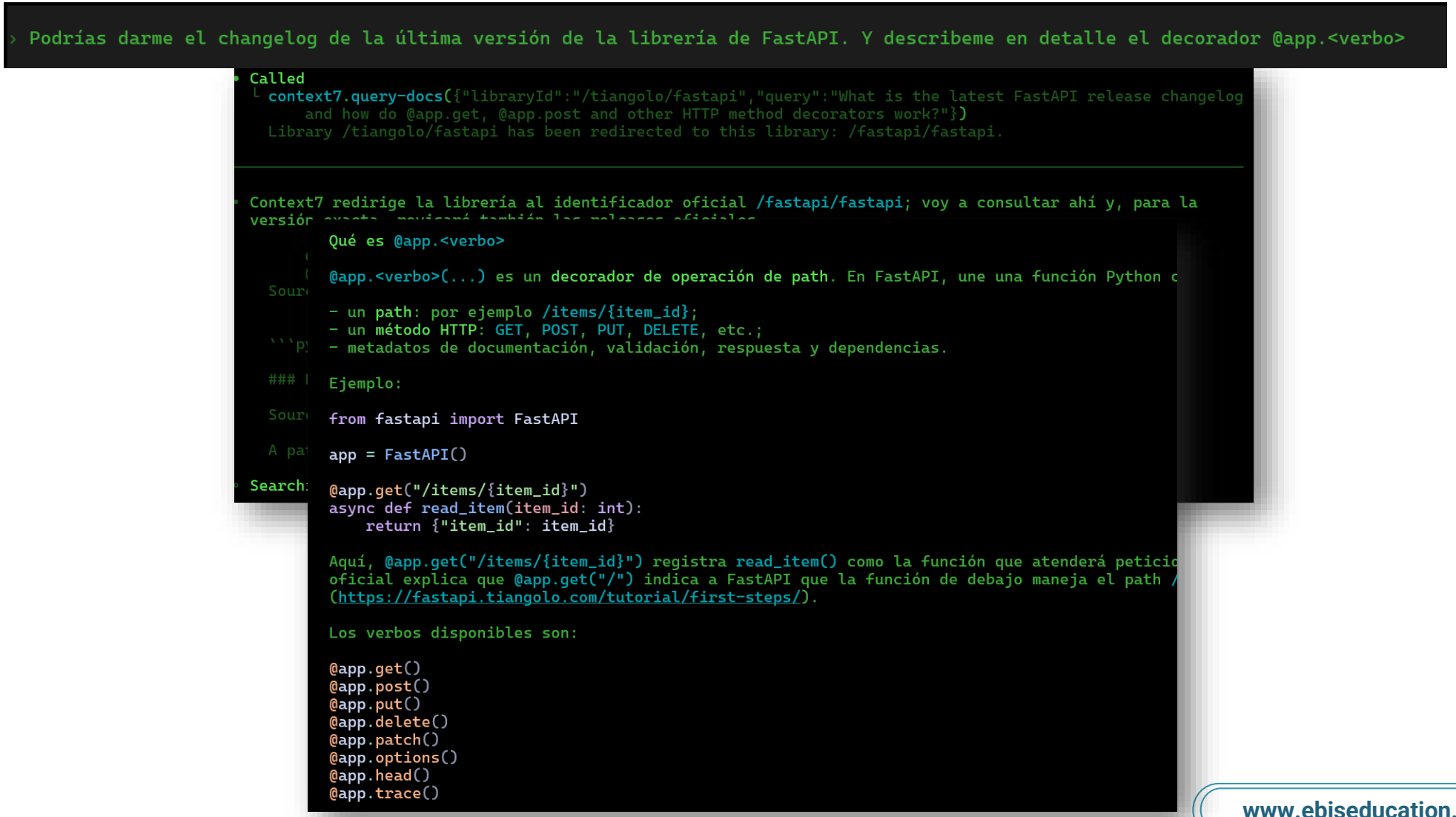
  - Actualización de traducciones update-outdated para: fr, ja, zh-hant, de, es, tr, pt, zh, uk, ko, ru.
  - Actualización de traducciones add-missing para: ko, es, zh-hant, pt, zh, ja, tr, uk, fr, de, ru.

  Cambios internos:

  - Actualización de páginas de FastAPI People: expertos, contribuidores y traductores.
  - Actualización de repositorios del topic de GitHub de FastAPI.
  - Limpieza de configuración de workflows.
  - Bumps de dependencias de GitHub Actions, paquetes Python y CodSpeedHQ/action.
```
```

MCPs Recomendados

EBIS Business Techschool



MCPs Recomendados

EBIS Business Techschool



MCPs Recomendados

Para configurarlo, **se añade el MCP Server en la configuración del Agente.**

En modo STDIO (se define el comando de para establecer la conexión local)




```
[mcp_servers.playwright]
command = "npx"
args = ["@playwright/mcp@latest"]
```

MCPs Recomendados

EBIS Business Techschool

```
› Puedes acceder a https://www.congreso.es/es/busqueda-de-diputados y filtrar por legislatura 10 y nombre "Pablo" y darme la lista de personas
  que sale como resultado.
```

```
• La versión de texto de la página solo muestra el formulario y se queda en “Cargando...”; voy a u
  JavaScript y leer los resultados reales.

• Calling playwright.browser_navigate({"url":"https://www.congreso.es/es/busqueda-de-diputados"})
```

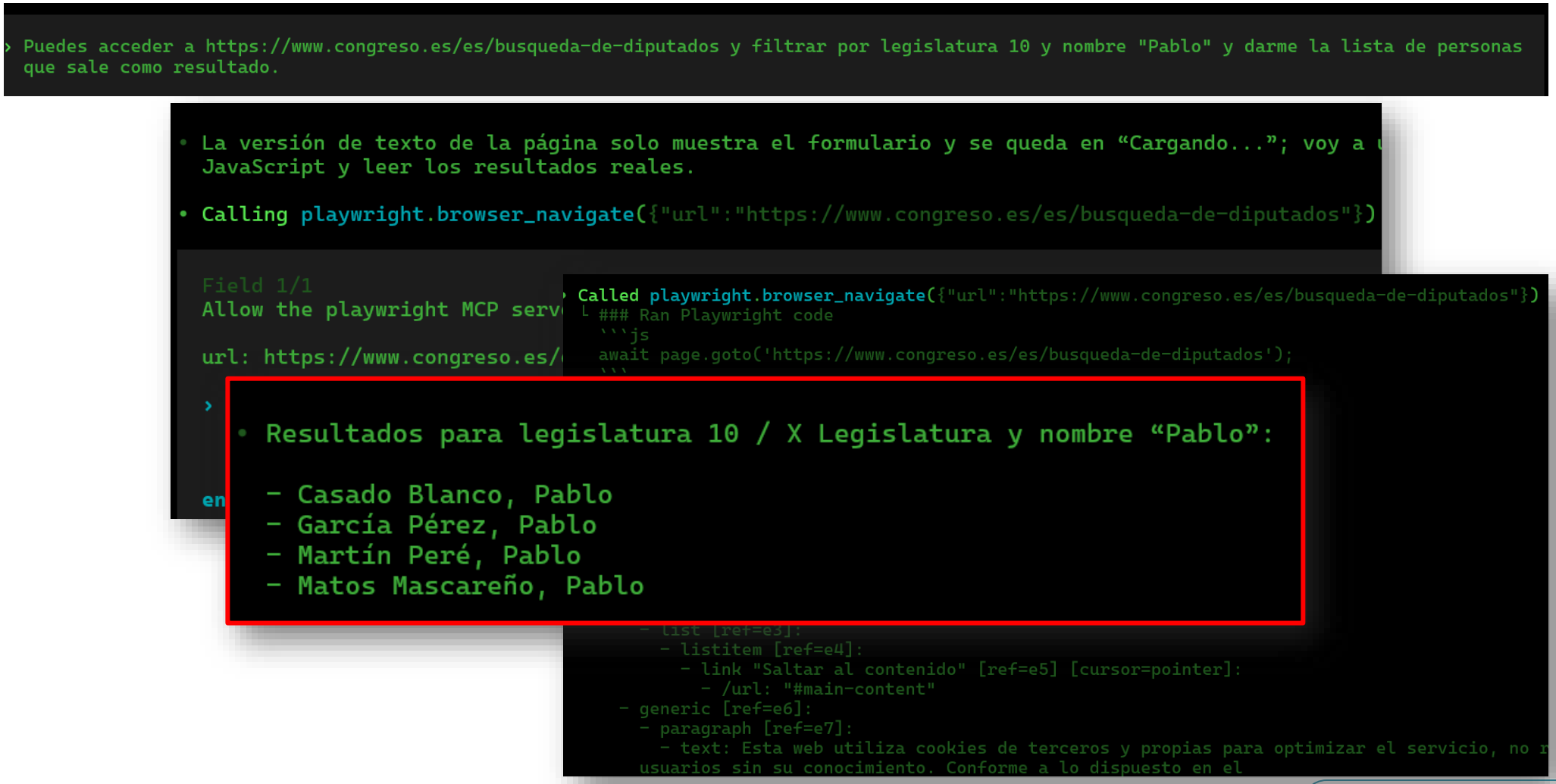
```
  Field 1/1
  Allow the playwright MCP serv

  url: https://www.congreso.es/

  › 1. Allow
    2. Allow for this session
    3. Always allow
    4. Cancel
  enter to submit | esc to canc
```

```
• Called playwright.browser_navigate({"url":"https://www.congreso.es/es/busqueda-de-diputados"})
  └ ### Ran Playwright code
    ```js
    await page.goto('https://www.congreso.es/es/busqueda-de-diputados');
    ```
    ### Page
    - Page URL: https://www.congreso.es/es/busqueda-de-diputados
    - Page Title: Búsqueda - Congreso de los Diputados
    - Console: 1 errors, 0 warnings
    ### Snapshot
    - [Snapshot](.playwright-mcp/page-2026-07-07T15-35-26-984Z.yml)
    - Page URL: https://www.congreso.es/es/busqueda-de-diputados
    - Page Title: Búsqueda - Congreso de los Diputados
    - Console: 1 errors, 0 warnings
    - generic [active] [ref=e1]:
      - navigation [ref=e2]:
        - heading "Búsqueda - Congreso de los Diputados" [level=1]
        - list [ref=e3]:
          - listitem [ref=e4]:
            - link "Saltar al contenido" [ref=e5] [cursor=pointer]:
              - /url: "#main-content"
      - generic [ref=e6]:
        - paragraph [ref=e7]:
          - text: Esta web utiliza cookies de terceros y propias para optimizar el servicio, no r
        usuarios sin su conocimiento. Conforme a lo dispuesto en el
```

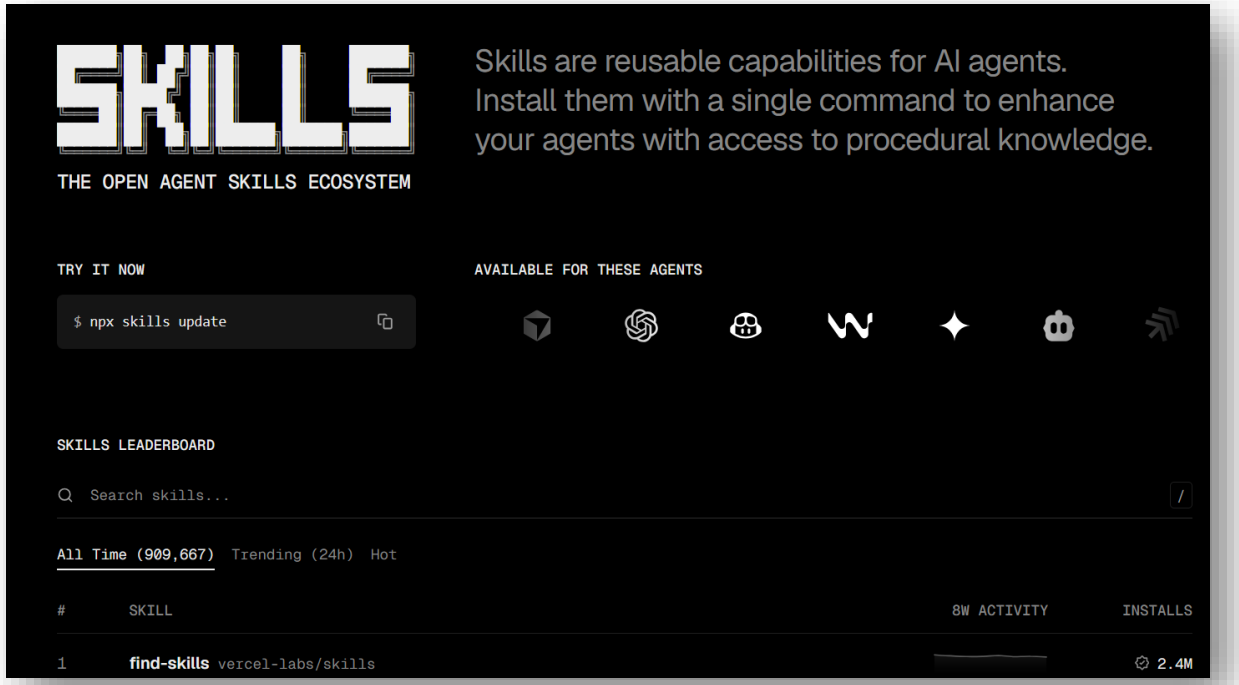
MCPs Recomendados



SKILLS.md

Documentación automática diseñada específicamente para que agentes de IA entiendan capacidades, flujos de trabajo y restricciones.

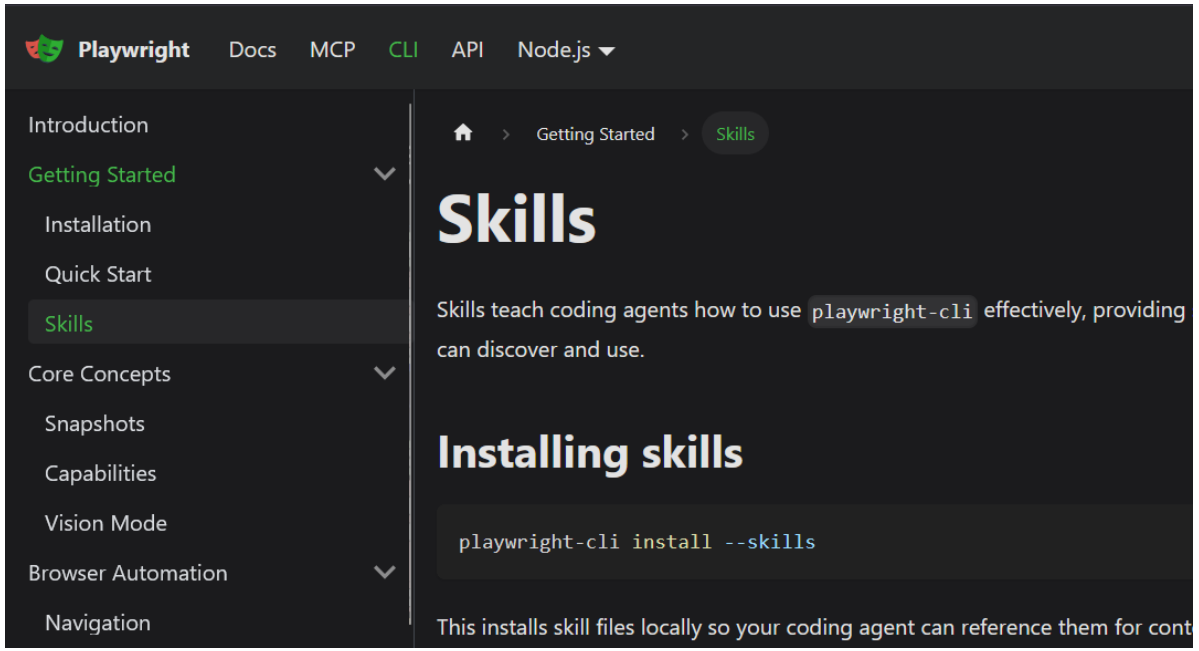
- **Contexto bajo demanda:** Elimina la necesidad de repetir instrucciones en cada conversación.
- **Estándar Técnico:** Formato Markdown estructurado.
- **Guía Operativa:** Define "cómo" debe actuar la IA ante herramientas externas.



skills.sh
(wiki de skills)

SKILLS.md

Si el **SKILLs.md** describe un CLI (**un comando**) instalado en el ordenador, gracias a la capacidad del agente de usar comandos, podrá usar la herramienta asimilando las capacidades de un **MCP (Model Context Protocol)**.



Playwright permite ser usado por los agentes sin MCP (con Skill)



EBIS Business Techschool

www.ebiseducation.com